

Trust-Gated Fast-Weight Updates for TTT-E2E Large Language Models

A runtime defense for language models that learn while they serve

TEAM

Manas Maahir

23BAI0051

Jay Krishna Kamlekar

23BAI0053

Kartheek Yadav

23BAI0056

| What this review covers

Three questions, in order

01

Motivation

What TTT-E2E is, the attack surface it opens, and why we picked this problem over the alternatives

02

What we have built

Literature survey, architecture decisions, a CPU-tested enforcement core, and the discipline around the numbers

03

What remains

Two hard gates ahead — the baseline check and the attack spike — then the gate prototype and evaluation

Ground rule for the whole project: every threshold in this deck was written down before the result it judges.

Background — the architecture we target

TTT-E2E: End-to-End Test-Time Training for Long Context · arXiv:2512.23675

- TTT-E2E reframes long-context modelling as **continual learning at inference**.
 - Instead of attending over a context, the model learns it.
- The model carries **fast weights θ** that it updates by next-token prediction as it reads the context.
 - The context is compressed into weights at near-RNN cost, while matching full-attention scaling.
- The base (**slow**) weights ϕ never move. θ moves **per request, at serve time**, inside the deployed system.
- The update point is a single vendor function: `MetaModel.inner_loop_step` — which is what makes a gate possible at all.

Why this matters for security

Every deployed safety property that assumes “the weights are the weights we shipped” stops being true the moment part of the model is written by the user’s own input stream.

Motivation — the attack surface this creates

Section 1 of 3 · Why the work exists

“A model that learns while it serves can be poisoned while it serves.”

The vector is a stream, not a token

A slow, benign-looking sequence of ordinary inputs — no malformed prompt, no obvious jailbreak, nothing a filter would flag — steers the fast-weight updates.

The damage lands somewhere else

Later, unrelated benign inputs get handled worse, or a latent trigger is left behind. Cause and effect are separated in time, so the harm is hard to attribute.

It is invisible to existing defenses

The base weights are never touched, so checkpoint scanning, training-data filtering and weight-integrity checks all pass. θ is not in any of them.

Consequence. If the harmful thing happens at update time, the check has to happen at update time. There is no offline stage left to put it in — which is exactly where our contribution sits.

Why we chose this problem

The gap our literature survey actually found

Research area	State of the art	Where that leaves us
Test-time poisoning attacks against TTA	Mature — IEEE S&P '23, ICLR '25, R.I.P. '24	Not our novelty. We inherit the threat model.
Defenses against test-time poisoning	MedBN (arXiv 2403.19326) exists — but it defends batch-norm statistics	Fast weights are not BN statistics. A BN-stat defense does not transfer.
Poisoning of TTT-E2E LLM fast weights	Untouched — the architecture is months old	Open. Demonstrating it is our Phase 1.
A gate built for fast-weight continual-inference updates	Untouched	Open. This is the contribution we are building.

Exactly one hard assumption

The whole idea rests on a single unproven claim — that the poisoning works. No stacked miracles.

The risky part is also the cheap part

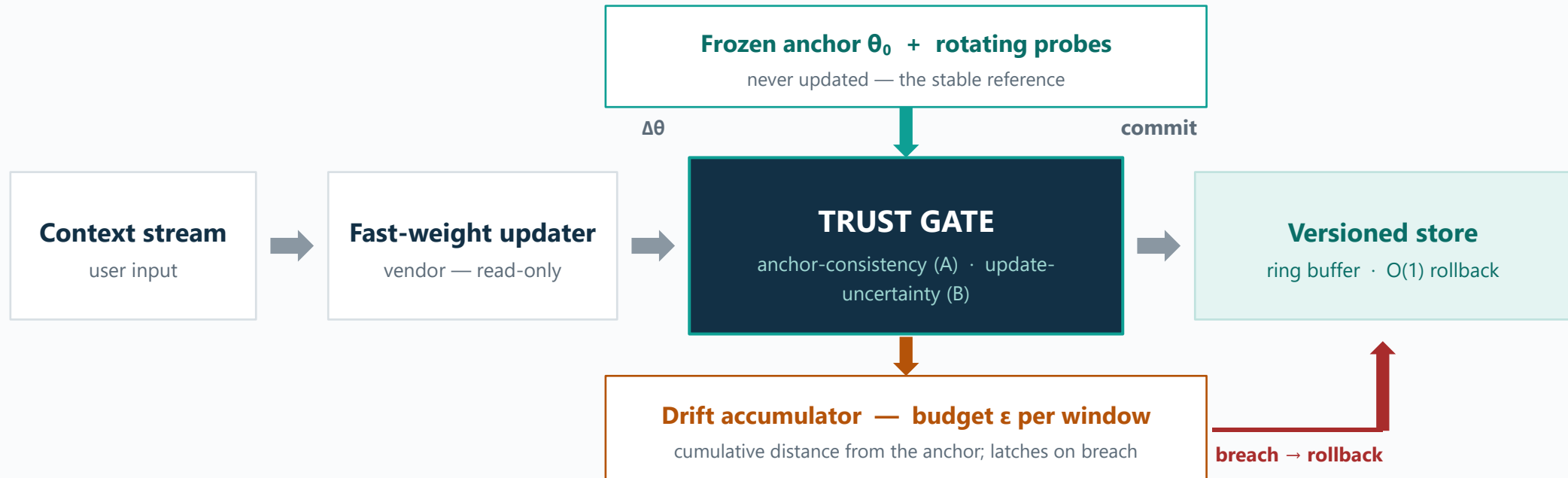
The attack demo both de-risks the idea and is the first thing we build. We find out early.

Buildable at our scale

Public code, a single 80 GB GPU, 1–3B parameters. No foundation-model training anywhere in the plan.

Our proposal — the trust gate

Intercept every proposed update; admit it only if it survives a check, a budget, and a rollback path



The headline property. Because JAX fast-weight updates are pure functions over a pytree, `reject` is a `jnp.where` select between weight trees and `rollback` is a reference swap — genuinely $O(1)$, branchless, with no mutable state to unwind. That is what turns “bounded drift” into a provable per-window bound rather than a hope.

What we have done · 1 of 5

Section 2 of 3 · Literature survey

- 15 peer-reviewed **journal** articles — IEEE Xplore and ScienceDirect, June 2024 onward. No preprints, no conference-only work.
- Every DOI resolved against **Crossref** and confirmed to redirect to the live publisher page.

Inference-time backdoors in LLM agents

Contextual backdoor attacks on embodied agents (TIFS '25) established that a crafted sequence of benign-looking inputs can implant a hidden backdoor at deployment — the exact threat model we defend.

Poisoning and defense in federated learning

Seven papers on model/data poisoning and robust aggregation. Nearest neighbour to our baseline — and the reason our MedBN-analogue is aggregation-based.

Robust adaptation and adversarial training

Adversarial self-training, contrastive graph defense, cloud-edge attack-defense. Shows what a defense costs on clean accuracy.

Graph and hypergraph poisoning

Momentum-gradient and node-injection attacks — useful for how attack strength is measured and reported, not for the mechanism.

What it produced. The gap statement on slide 5 — the attack literature is mature, a defense exists but it guards batch-norm statistics, and nothing addresses fast weights. That is the sentence the whole project hangs on.

What we have done · 2 of 5

Architecture decisions, each recorded as an ADR

ADR-002

Overlay, not a fork

Upstream TTT-E2E ships with no license. It is vendored as a read-only pinned submodule (SHA a4fc478) and never edited; all our code imports and wraps it.

ADR-003

One enforcement point

The gate hooks `MetaModel.inner_loop_step` — the single place a fast-weight update commits — through a confined, reversible monkeypatch.

ADR-004

The eval entry point is real

Resolved that the vendor exposes an eval path (`training.eval_mode=true` on train). This was the hard unknown gating the cost model and the bootstrap script.

ADR-005

The baseline is a soundness check

The paper publishes no loss number for the released 1B checkpoint. So Phase 0.5 verifies our environment, and we say so — it is not a reproduction.

Why the overlay constraint is a feature. It forced every design choice to be expressible as a wrapper around code we cannot change. That is also the honest deployment story: a defense that requires forking the model is a defense nobody installs.

What we have done · 3 of 5

The enforcement core — built and tested on CPU

Component	State	Verified by
Update interceptor + jit-safe pytree ops	BUILT	accept / reject / no-op semantics; passthrough identity
Drift accumulator — latching budget ϵ	BUILT	latch fires once and stays fired; trace-safe
Versioned store — ring buffer, $O(1)$ rollback	BUILT	round-trip and rollback to an arbitrary depth
Corruption metrics, audit log, report renderer	BUILT	Cohen's d, relative degradation, attack-success rate
Attack module + model-touching harness	SPECIFIED	interfaces exist and raise; blocked on checkpoints
Gate signals A / B / C + MedBN-analogue baseline	PHASE 2	gated on the Phase 1 result — deliberately unwritten

31

CPU tests passing
no GPU, no checkpoints

The gap we record rather than hide

The accumulator and the store are implemented but not yet wired into the interceptor. So today the bounded-drift guarantee is designed, not enforced — written into ADR-003 as a dated correction, and scheduled as the project's highest-risk task.

What we have done · 4 of 5

Fixing the bar before there is a number to judge

Pre-registered kill-gate

Frozen 2026-07-28, before any attack code ran and committed to git so the history proves the bar predates the result:
Cohen's $d \geq 0.8$ AND relative degradation $\geq 10\%$ AND fluency ratio ≤ 1.5 , over 5 seeds, with the most realistic attack strategy as the headline.

Pre-registered baseline tolerance

Two-sided band $2.314 < \text{loss} < 2.805$ nats/token plus four structural checks, derived from the paper's figure data and cross-validated across all 35 values.

Why we did it this way

A go/no-go rule written after seeing the result is not a rule. Freezing the threshold first is the only thing that makes a STOP verdict credible — including to ourselves.

Cost model

Checkpoint ≈ 5.9 GB, expected spend \$10–35, hard cap \$325 for Phase 0.5. Found and priced the multi-GPU cliff at 3B.

Cold-start bootstrap

One idempotent script takes a bare GPU box to a ready-to-paste eval command.

Egress guard

A metadata-only probe prints the byte count before any transfer, and a cap refuses rather than surprises.

Working protocols

GPU queue and booking ledger, branch-and-review rules. One box, three people — someone has to hold the calendar.

What we have done · 5 of 5

Written output and reproducibility

P2 deliverable — Introduction + Related Work

IEEEtran manuscript. Thematic organisation of the field, key representative studies, strengths and limitations, the research gap, and where this work sits in it.

P3 deliverable — + Proposed Methodology

≈1,600 lines, self-contained, compiles on a stock IEEEtran install. Threat model, crafted-stream construction, gate architecture, both scoring signals, the drift accumulator with a bounded-drift proposition, the versioned store, decision policy, complexity and overhead, baselines, and the evaluation protocol.

Architecture decision record

Five ADRs, including a dated correction to one of them

Repository knowledge graph

Generated map of every module and its dependencies

Roadmap + team execution plan

Phase gates, task ownership, acceptance criteria, risk register

What remains · the two gates ahead

Section 3 of 3 · Neither can be skipped

PHASE 0.5 · blocked on hardware

Baseline environment check

- Procure a Weights & Biases key — the only Phase 0.4 item still open
- Book the box in the ledger before it starts billing
- Probe the validation-split size first — that request is free
- Rehearse on the 125M model in the same session
- Run the vendor eval unmodified on 1B @ 8K, gate provably not installed
- Read the pre-registered band before reading our own number

PHASE 1 · THE KILL-GATE

Attack spike

- Implement the attack objectives — degrade and trigger
- Build the three stream strategies plus a length-matched benign control
- Write the crafting loop, fluency term inside the objective
- Wire the model-touching half of the evaluation harness
- Run five seeds in both arms, poison and control
- Print observed against pre-registered — and take the verdict

The one rule that governs everything. If no benign-looking stream measurably corrupts fast weights against the pre-registered threshold, the project stops. No demonstrated attack means there is no defense worth building. This discipline is the point, not an obstacle.

What remains · if the kill-gate opens

Phases 2 and 3 are deliberately unwritten until then

PHASE 2

Gate prototype and evaluation

- Anchor-consistency signal — probe divergence against the frozen anchor
- Update-uncertainty signal — the cheap secondary check
- Rotating held-out probes at fixed shape, to avoid recompilation
- Decision policy with every threshold as a parameter, none hard-coded
- Influence estimation as an offline auditor, to give the curves a ground truth
- A MedBN-analogue baseline — and we tune it hard, because beating a weak baseline proves nothing
- Operating-point sweep: ROC, clean-accuracy regression, $\leq 10\%$ compute overhead, store memory

THE HARDEST OPEN PROBLEM

Carrying drift state through the vendor's scan

The vendor's scan carry is fixed, and we may not edit vendor code — a scan's carry structure must come out exactly as it went in, so our wrapper cannot just add a slot. Three routes out, none obviously right, and one of them quietly weakens what we can claim. It gets a written ADR before anyone writes code.

PHASE 3

Novelty validation

Line-level differencing against MedBN and the robust-TTA literature, then the bounded-drift proof and the measured results.

Note. None of this is stubbed-out optimism — the interfaces exist and every one of them raises, on purpose, so that nothing downstream can quietly run on a placeholder.

Risks we are tracking

Each one has a written trigger and a written response

Risk	What we would see	What we do about it
The baseline misses the band	Loss outside 2.314–2.805, or the per-token NLL does not fall	Stop everything and debug the environment. Nothing downstream is interpretable until it passes.
Only the unrealistic attack works	The realistic strategy shows near-zero effect size after two or three seeds	Report it honestly as a weaker threat model. The STOP option stays live — that is what pre-registration is for.
The scan cannot carry per-window drift state	All three carry-state routes look bad on paper	Escalate before coding. If only the weakest route works, restate the guarantee at the granularity we actually deliver.
The validation split is larger than modelled	The free metadata probe returns a byte count far above the estimate	One free command settles it before any billing starts. The cap refuses rather than surprises.

The common thread. Every row was written before the situation it describes. A risk register written after the fact is just a changelog.

Where we stand

Summary

Phase 0

Foundation

DONE

Phase 0.4

Lead prep package

DONE

Phase 0.5

Baseline check

BLOCKED

Phase 1

Attack spike

KILL-GATE

Phase 2

Gate prototype

GATED

Phase 3

Novelty validation

GATED

31

CPU tests green

15

journal papers surveyed

5

architecture decisions recorded

0

numbers produced on a GPU

Everything that can be decided at a desk has been decided.

What remains needs the box — and an honest answer to one question: does the attack actually work?